



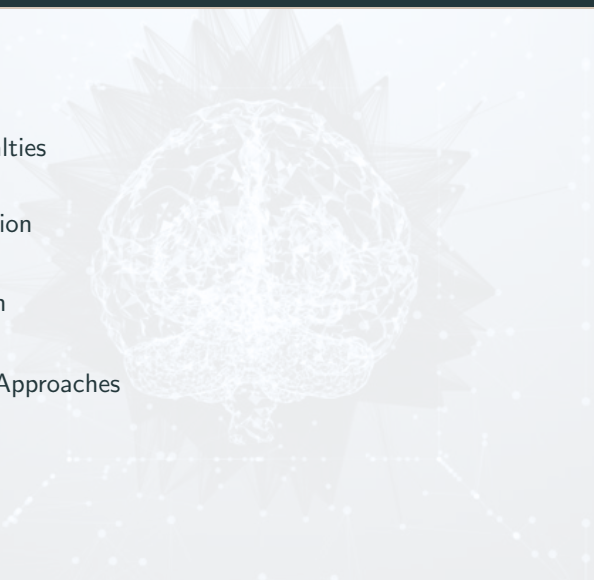
Deep Learning

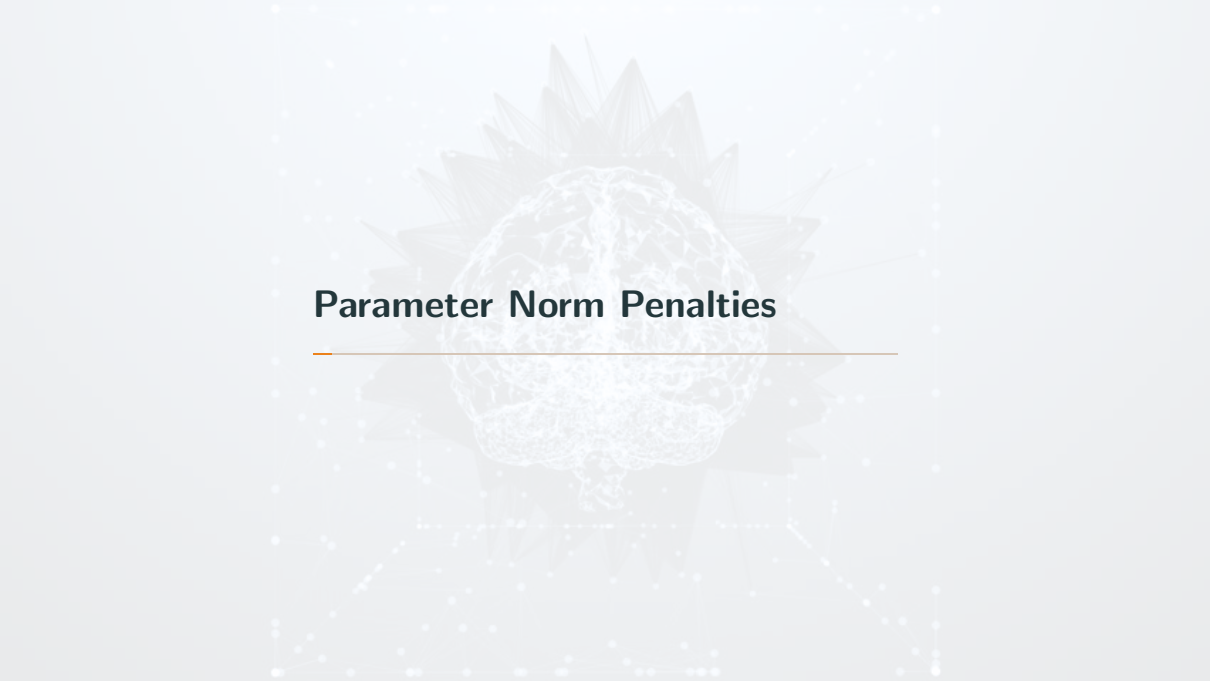
Regularization

Lecturer: Duc Dung Nguyen, PhD.

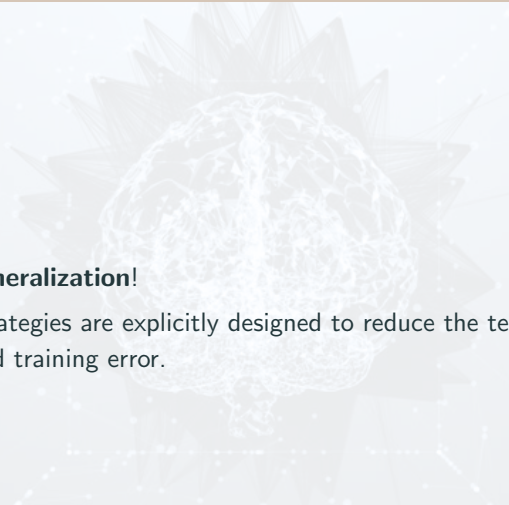
Contact: nddung@hcmut.edu.vn

Faculty of Computer Science and Engineering
Hochiminh city University of Technology

- 
- A large, faint, stylized graphic of a brain with a network of white lines and dots, resembling a neural network or a complex data structure, set against a light gray background with a subtle grid of dots.
1. Parameter Norm Penalties
 2. Constrained optimization
 3. Dataset Augmentation
 4. Other Regularization Approaches
 5. Dropout



Parameter Norm Penalties

- 
- A faint, stylized background graphic of a brain with neural connections, overlaid on a network of dots and lines.
- Problem in ML: **generalization!**
 - **Regularization**: strategies are explicitly designed to reduce the test error, possibly at the expense of increased training error.

- 
- A decorative background image featuring a stylized, light-colored lotus flower or similar floral motif, overlaid with a network of white dots and lines, suggesting a neural network or data structure.
- Most regularization strategies are based on regularizing estimators
 - Regularization of an estimator works by trading *increased bias for reduced variance*
 - **An effective regularizer:** makes a profitable trade, reducing variance significantly while not overly increasing the bias

- Main regularization approaches: limiting the capacity of the model by adding a parameter norm penalty $\Omega(\theta)$ to the objective function J

$$\tilde{J}(\theta, \mathbf{X}, \mathbf{y}) = J(\theta, \mathbf{X}, \mathbf{y}) + \alpha\Omega(\theta) \quad (1)$$

- Different choices of the parameter norm Ω can result in different solutions being referred
- In NNs, Ω is chosen to penalize *only the weights* of the affine transformation at each layer (leave the bias unregularized)

- The most common parameter norm penalty: L^2 (**weight decay**), also called **ridge regression**, or **Tikhonov regularization**

$$\tilde{J}(\mathbf{w}, \mathbf{X}, \mathbf{y}) = J(\mathbf{w}, \mathbf{X}, \mathbf{y}) + \alpha \Omega(\mathbf{w}) = J(\theta, \mathbf{X}, \mathbf{y}) + \frac{\alpha}{2} \mathbf{w}^\top \mathbf{w} \quad (2)$$

with the corresponding parameter gradient

$$\nabla_w \tilde{J}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = \alpha \mathbf{w} + \nabla_w J(\mathbf{w}; \mathbf{X}, \mathbf{y}) \quad (3)$$

- Gradient step

$$\mathbf{w} \leftarrow \mathbf{w} - \epsilon(\alpha \mathbf{w} + \nabla_w J(\mathbf{w}; \mathbf{X}, \mathbf{y})) = (1 - \epsilon\alpha) \mathbf{w} - \epsilon \nabla_w J(\mathbf{w}; \mathbf{X}, \mathbf{y}) \quad (4)$$

- L^1 regularization

$$\Omega(\theta) = \|\mathbf{w}\|_1 = \sum_i |w_i| \quad (5)$$

- Regularized objective function

$$\tilde{J}(\mathbf{w}, \mathbf{X}, \mathbf{y}) = J(\mathbf{w}, \mathbf{X}, \mathbf{y}) + \alpha \|\mathbf{w}\|_1 \quad (6)$$

with the corresponding parameter gradient

$$\nabla_w \tilde{J}(\mathbf{w}; \mathbf{X}, \mathbf{y}) = \alpha \text{sign}(\mathbf{w}) + \nabla_w J(\mathbf{w}; \mathbf{X}, \mathbf{y}) \quad (7)$$

- The regularization contribution to the gradient no longer scale linearly with each w_i
- L^1 regularization results in a solution that is more **sparse**, comparing to L^2 : some parameters have an optimal value of zero.
- Sparsity property of L^1 regularization has been used extensively as a **feature selection** mechanism

Constrained optimization

Constrained optimization

- Find the maximal or minimal value of $f(x)$ for values of x in some set $\mathbb{S}$
- **Feasible points:** points x that lie within the set $\mathbb{S}$
- Find a solution that is small in some sense
- Common approach: impose a norm constraint, such as $\|x\| \leq 1$

Approach to constrained optimization

- Modify gradient descent taking the constraint into account
- If we use a small constant step size ϵ , we can make gradient descent steps, then project the result back into $\mathcal{S}$.
- If we use a line search, we can search only over step sizes ϵ that yield new $\mathbf{x}$ points that are feasible, or we can project each point on the line back into the constraint region.

Karush–Kuhn–Tucker (KKT): a very general solution to constrained optimization.

- **KKT multipliers:** introduce new variables λ_i and α_j for each constraint
- The generalized Lagrangian is then defined as

$$L(\mathbf{x}, \lambda, \alpha) = f(\mathbf{x}) + \sum_i \lambda_i g^{(i)}(\mathbf{x}) + \sum_j \alpha_j h^{(j)}(\mathbf{x}). \quad (8)$$

- Solve a constrained minimization problem using unconstrained optimization of the generalized Lagrangian
- Minimize $\min_{\mathbf{x} \in \mathcal{S}} f(\mathbf{x})$ is equivalent to

$$\min_x \max_{\lambda} \max_{\alpha, \alpha \geq 0} L(x, \lambda, \alpha). \quad (9)$$

- This follows because any time the constraints are satisfied,

$$\max_{\lambda} \max_{\alpha, \alpha \geq 0} L(\mathbf{x}, \lambda, \alpha) = f(\mathbf{x}) \quad (10)$$

while any time a constraint is violated

$$\max_{\lambda} \max_{\alpha, \alpha \geq 0} L(\mathbf{x}, \lambda, \alpha) = \infty \quad (11)$$

- *No infeasible point will ever be optimal*
- The optimum within the feasible points is unchanged

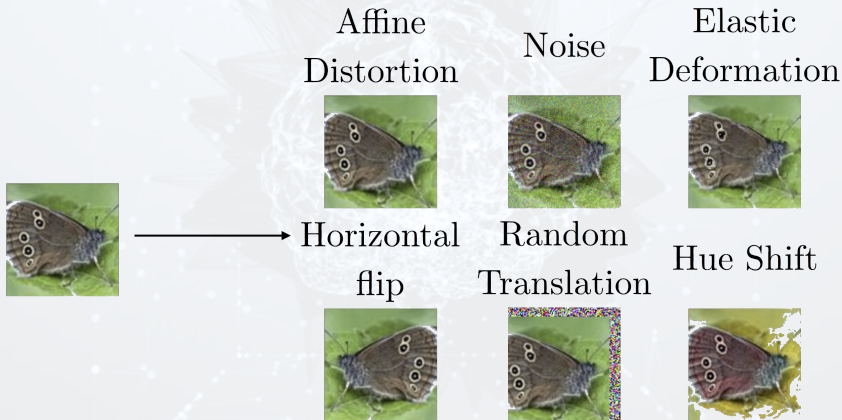
Dataset Augmentation



- Making ML model generalize better: train on more data!
- In practice: **the amount of data is limited!**
- Solution: *create fake data for training*

- Dataset augmentation: a particularly effective technique for **object recognition**
- Image: high dimensional, enormous variety of factors of variation
- E.g.: rotating, scaling, affine transformation, etc.
- Dataset augmentation is effective for **speech recognition** tasks as well

- Injecting noise
- NN prove *not to be very robust to noise*
- Unsupervised learning: **denoising autoencoder**
- Noise in hidden units: dataset augmentation at multiple levels of abstraction

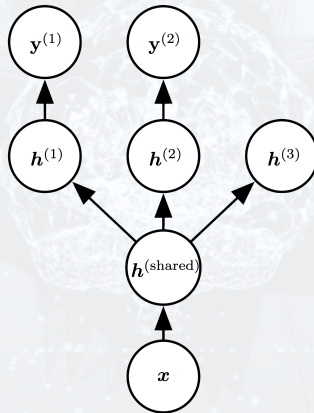




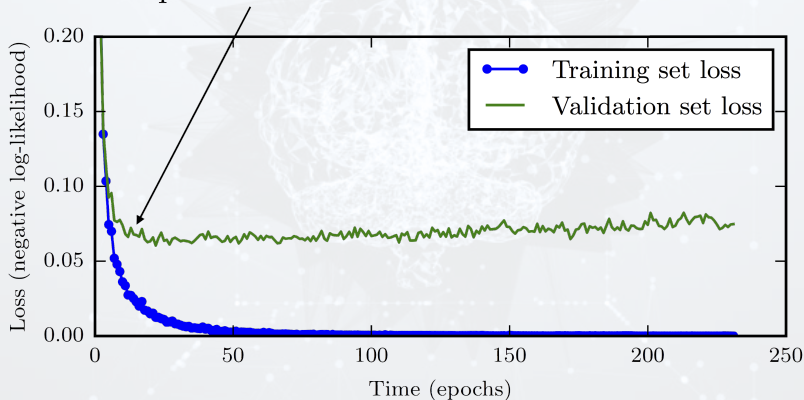
Other Regularization Approaches

- **Semi-supervised learning**: usually refers to learning a representation $h = f(\mathbf{x})$
- Learn a representation so that examples from the same class have similar representations
- Provide useful cues for how to group examples in representation space
- A linear classifier in the new space may achieve better generalization in many cases
- **Principal components analysis (PCA)**: a pre-processing step before applying a classifier (on the projected data)

- Construct models in which a generative model of either $P(\mathbf{x})$ or $P(\mathbf{x}, \mathbf{y})$ shares parameters with a discriminative model of $P(\mathbf{y}|\mathbf{x})$
- *Trade-off* the supervised criterion $-\log P(\mathbf{y}|\mathbf{x})$ with the unsupervised or generative one (such as $-\log P(\mathbf{x})$ or $-\log P(\mathbf{x}, \mathbf{y})$)
- The generative criterion expresses a particular form of *prior belief* about the solution to the supervised learning problem



Early stopping: terminate while validation set performance is better



- **Parameter sharing:** force sets of parameters to be equal
- Interpret the various models or model components as sharing a unique set of parameters.
- Only a subset of the parameters (the unique set) need to be stored
- **CNN**

Dropout



- **Bagging (bootstrap aggregating)**: a techniques to reduce generalization error by combining several models (Breiman, 1994)
- General strategy in ML: **model averaging** → **ensemble method**

Original dataset



First resampled dataset



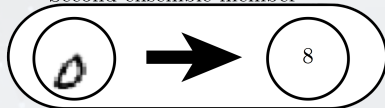
First ensemble member

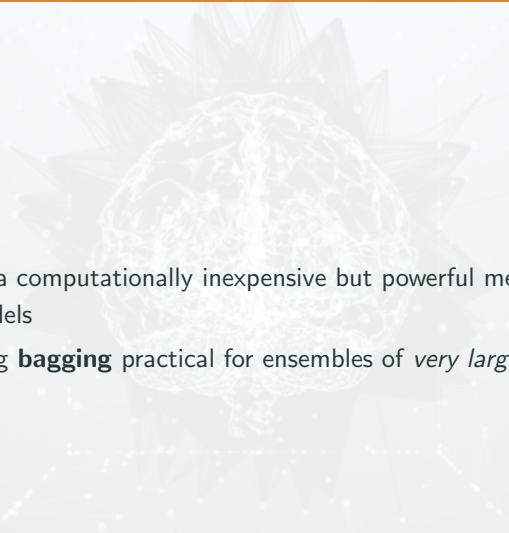


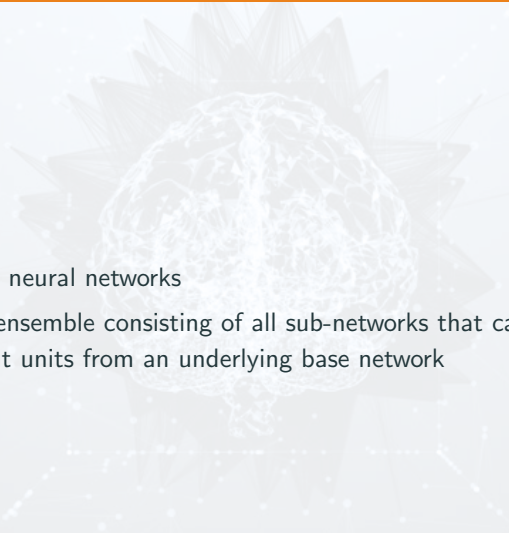
Second resampled dataset

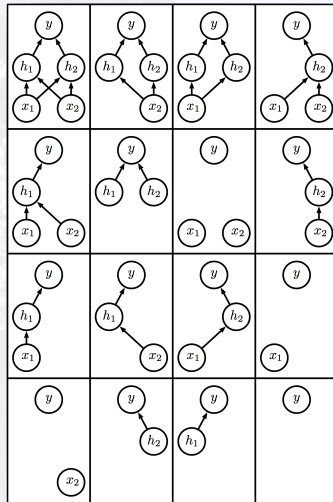
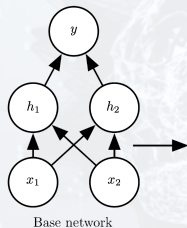


Second ensemble member



- 
- A faint, stylized background graphic of a human brain with a network of white lines representing neural connections, set against a light gray background.
- **Dropout:** provides a computationally inexpensive but powerful method of regularizing a broad family of models
 - A method of making **bagging** practical for ensembles of *very large neural networks*.

- 
- A faint, stylized background image of a human brain with a network of white lines representing neural connections. The brain is centered and has a spiky, star-like outline. The lines connect various points across the brain's surface and extend outwards.
- Good for five to ten neural networks
 - Dropout trains the ensemble consisting of all sub-networks that can be formed by removing non-output units from an underlying base network



Ensemble of subnetworks

- **Bagging**

- The models are independent
- Each model is trained to convergence on its respective training set

- **Dropout**

- Models share parameters
 - most models are not explicitly trained at all
 - It is *infeasible* to sample *all possible subnetworks* within the lifetime of the universe
 - The remaining sub-networks to arrive at good settings of the parameters
- Dropout can *represent an exponential number of models* with a tractable amount of memory

- Assume that the model's role is to output a probability distribution
- Bagging:
 - each model i produces a probability distribution $p^{(i)}(y|x)$
 - The prediction of the ensemble is given by the arithmetic mean of all of these distributions

$$\frac{1}{k} \sum_{i=1}^k p^{(i)}(\mathbf{y}|\mathbf{x}) \quad (12)$$

- Dropout:
 - Each sub-model defined by mask vector μ defines a probability distribution $p(\mathbf{y}|\mathbf{x}, \mu)$
 - The arithmetic mean over all masks is given by

$$\sum_{\mu} p(\mu) p(\mathbf{y}|\mathbf{x}, \mu) \quad (13)$$

where $p(\mu)$ is the probability distribution that was used to sample μ at training time

- **Very computationally cheap**
- Using dropout during training requires only $O(n)$ computation per example per update, to generate n random binary numbers and multiply them by the state
- Dropout does not significantly limit the type of model or training procedure that can be used

- The cost of using dropout in a complete system *can be significant*
- *Increase the size* of the model
- Typically the *optimal validation set error is much lower* when using dropout
- The cost: **a much larger model** and many **more iterations** of the training algorithm

- For **very large datasets**
 - Regularization confers little reduction in generalization error
 - The computational cost may outweigh the benefit of regularization
- **Very few data samples**
 - Dropout is less effective
 - When additional unlabeled data is available, unsupervised feature learning can gain an advantage over dropout